



Universidad
Rey Juan Carlos

HIGHER TECHNICAL SCHOOL OF COMPUTER ENGINEERING
MASTER'S DEGREE IN COMPUTER GRAPHICS, GAMES, AND VIRTUAL
REALITY

2025-2026 Academic Year

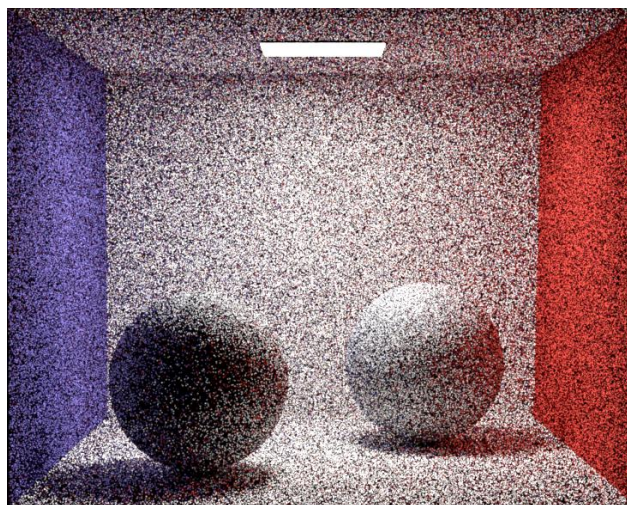
Advanced Rendering I

Practice 3

Carlos Escribano González

Path Tracer with Indirect Lighting:

The first part of the practice requires implementing a brute-force path tracer that computes the global illumination of the scene by tracing rays that bounce off objects until an emitter is reached, without directly sampling the lights. To this end, the implementation was carried out in *path.cpp* through the recursive *LiRecursive()* function, which uses a *PathInfo* struct to store information between iterations. Thus, whenever a ray bounces off an object in the scene, it checks whether the object is an emitter. If it is, the emitted color is multiplied by the accumulated throughput from the previous bounces and returned. Otherwise, the path throughput is updated, and a new ray is generated, restarting the same process. For the *cbox* scene, with 32 samples per pixel and Lambertian materials, the result obtained in 8.4 seconds is shown below:

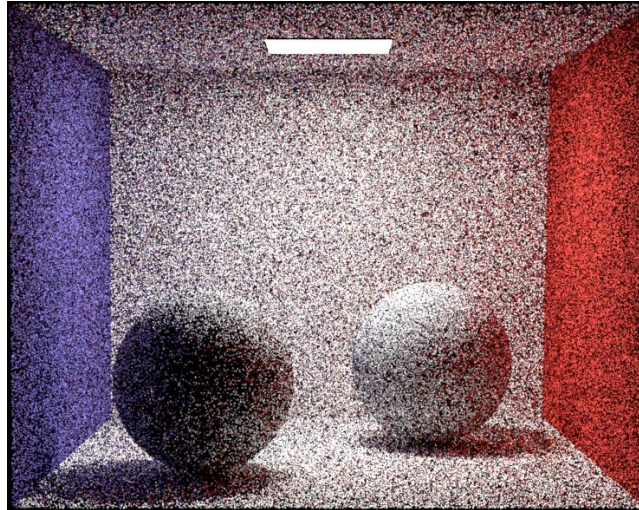


Global illumination path tracer without NEE or Russian roulette; 32 spp

As can be seen in the image, the large amount of noise demonstrates the need to directly sample the lights at each ray bounce, i.e., to implement *Next Event Estimation*. However, before doing so, Russian roulette was implemented. This technique allows rays to be terminated prematurely, preferably when their remaining energy is low. In this way, the convergence of the algorithm can be accelerated with little effect on the variance of the estimator.

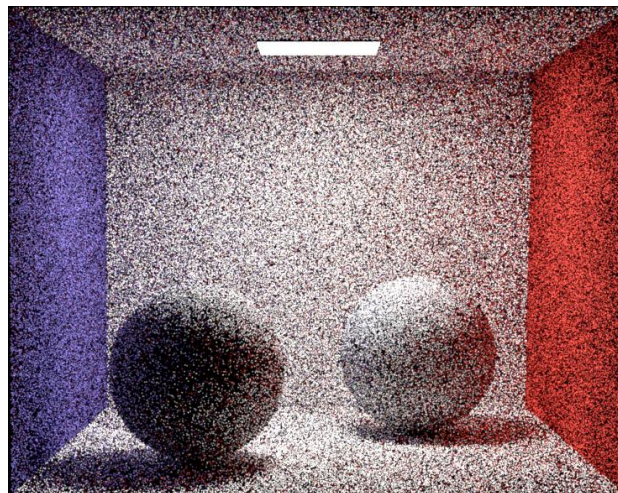
For this purpose, the *russianRoulette()* method was implemented. It returns a random value corresponding to the probability of terminating the ray and updates the throughput by dividing it by the corresponding survival probability. If a random number is lower than the probability of terminating the ray, the recursive algorithm returns black.

With Russian roulette implemented, results visibly similar to those obtained previously could be achieved in slightly less time. With a fixed Russian roulette termination probability of 0.05, under the same conditions as the previous render, the following result was obtained in 6.1 seconds:



Russian Roulette with a fixed value of 0.05; 32 spp

However, if the probability of terminating the ray is modeled according to its remaining energy, in this case, based on the maximum component of the throughput, the result can be obtained in even less time while remaining virtually identical, as shown by the following render, obtained in 5.6 seconds:



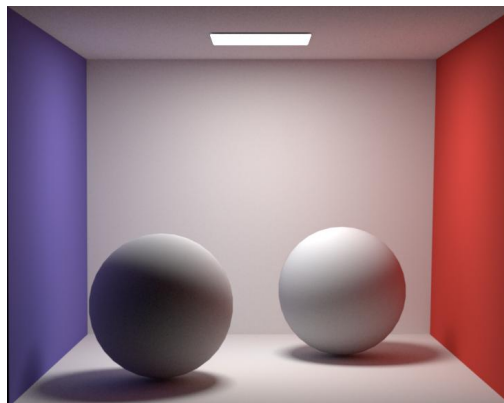
Russian roulette based on the remaining throughput energy; 32 spp

Path Tracer with Direct and Indirect Lighting:

In the second part of the practice, *Next Event Estimation* was added to the previous implementation, i.e., direct light sampling at every bounce. Therefore, at each intersection, an indirect-lighting ray is generated, which continues the path described in the previous section, as well as a direct-lighting ray, which is not traced recursively.

To achieve this, the functionality of *path.cpp* was copied to *path_nee.cpp* and the necessary modifications were made. First, before updating the throughput and generating the next ray, the direct lighting contribution is obtained using NEE by tracing a ray towards an emitter in the scene, as was done in Practice 2. Additionally, when a ray is terminated by Russian roulette, the algorithm returns the direct-lighting contribution calculated previously instead of black, since direct lighting, unlike indirect lighting, does not depend on continuing the path.

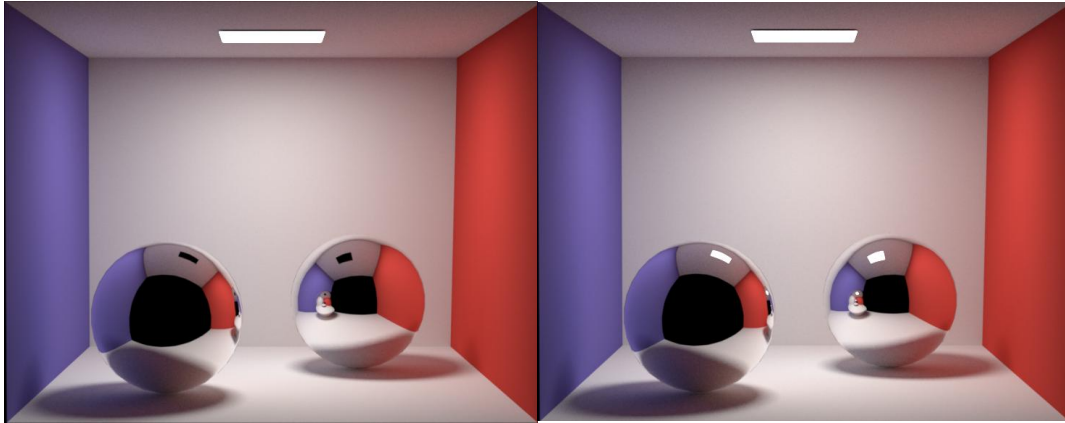
Since the two generated rays, direct and indirect, integrate different parts of the rendering equation, they can be treated as independent paths. Therefore, the value returned by each recursive call is the sum of the corresponding contributions, without the need to perform MIS between them. However, this possibility of simply adding the two contributions disappears when the final indirect-lighting bounce reaches an emitter. In that case, since both paths represent the same event, MIS must be applied to the indirect ray to reduce the variance of the estimator.



Path tracer with NEE; 512 spp

Much better results can be observed compared with those previously obtained without NEE. Furthermore, compared with the results from Practice 2, a much more natural result is achieved, introducing natural lighting phenomena such as ambient occlusion and color bleeding while reducing the amount of noise.

It is worth noting that when the material intersected by the ray is a perfect mirror, the PDF of the indirect sampling must be set to 1 before performing MIS; otherwise, reflections of the emitters appear black:



Perfect mirrors without PDF adjustment (left) vs. with PDF adjustment (right); 512 spp

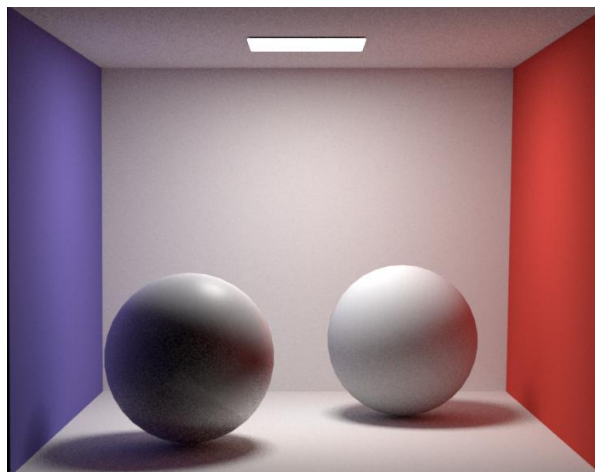
Microfacet-Based BRDF:

As an optional extension, a microfacet-based BRDF was implemented in *microfacet.cpp*. This file implements the functions defined by the Cook–Torrance model and expanded upon in the paper by Walter et al.: the *Normal Distribution Function* (D), *Fresnel* (F), and *Geometry / Shadow-Masking* (G) terms. This made it possible to implement, in the *eval()* function, both the diffuse (Lambertian) and specular (microfacet) components.

Subsequently, in the *sample()* method, the outgoing ray direction must be selected according to the material properties. It can be sampled diffusely, in which case *squareToCosineHemisphere()* is used, or specularly, in which case the incoming ray is reflected with respect to the half-vector h , which is computed according to the orientation of the microfacets.

Finally, *pdf()* must return the probability of having selected the given ray based on the probability of selecting each lobe and the probability that the selected lobe generates the corresponding ray.

The resulting image, using two spheres with different specular components, is shown below:



Spheres with a microfacet-based BRDF; 128 spp

Experiments:

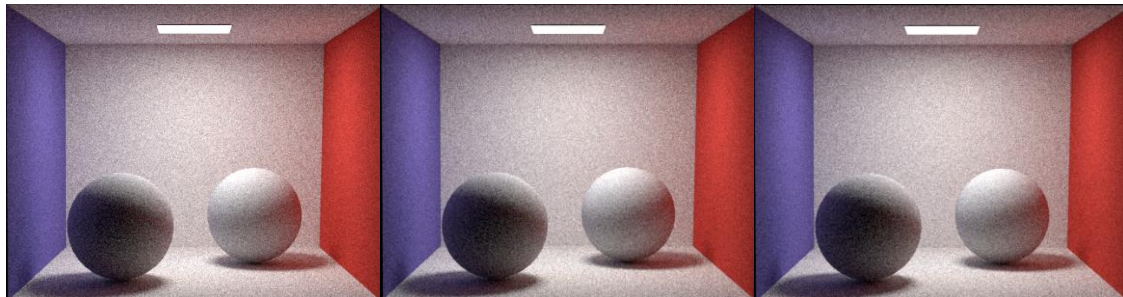
Experiment 1 – Convergence with Russian Roulette

As explained above, differences in the convergence time of the algorithm can be observed depending on the threshold selected for terminating rays using Russian roulette. This difference in rendering time becomes more pronounced as the number of samples per pixel increases, as shown in the following table:

	Without Russian roulette	Fixed Russian roulette (0.05)	Adaptive Russian roulette
32 spp	8.4 seconds	6.1 seconds	5.6 seconds
64 spp	16.8 seconds	12.4 seconds	11.1 seconds
128 spp	33.2 seconds	24.9 seconds	22.5 seconds
256 spp	66 seconds	49.8 seconds	44.3 seconds
512 spp	132 seconds	102 seconds	90 seconds

Although the difference is barely noticeable for a low number of samples per pixel, when reaching values such as 512 spp, the improvement exceeds 30%, corresponding to a difference of more than 40 seconds in the evaluated case.

This demonstrates that Russian roulette is a useful technique for reducing rendering time without affecting visual quality, as shown by the following 512 spp renders generated using the implementation without NEE, whose convergence times are given in the table above:



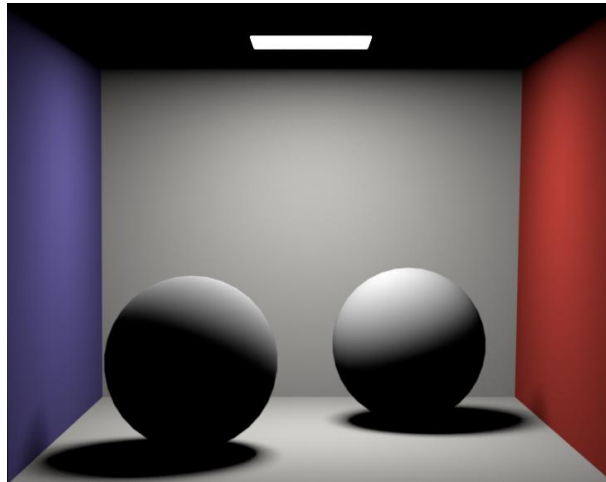
Without Russian roulette (left) vs. fixed Russian roulette (center) vs. adaptive Russian roulette (right); 512 spp

Experiment 2 – Natural vs. Explicit Ambient Occlusion

In the second practice of the course, a “manual” calculation of ambient occlusion was implemented. In that implementation, found in *direct.cpp*, a ray is cast from the evaluated point with a predefined maximum distance. If it intersects any object in the scene, the function returns 0; otherwise, it returns 1. This value is multiplied by the color calculated by *Li()*, making pixels darker according to how occluded they are.

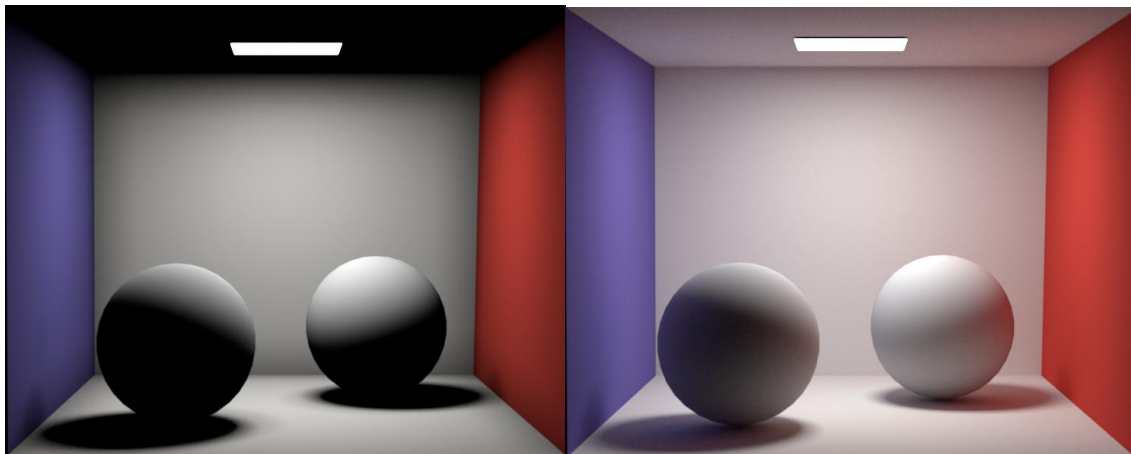
However, the path tracer developed for this third practice already includes ambient occlusion naturally, since the recursive algorithm itself causes occluded regions to appear darker. In other words, ambient occlusion emerges naturally from the rendering equation.

The scene without ambient occlusion, using the integrator from the previous practice, would look like this:



Scene with direct lighting, without ambient occlusion

With ambient occlusion, the result is as follows:

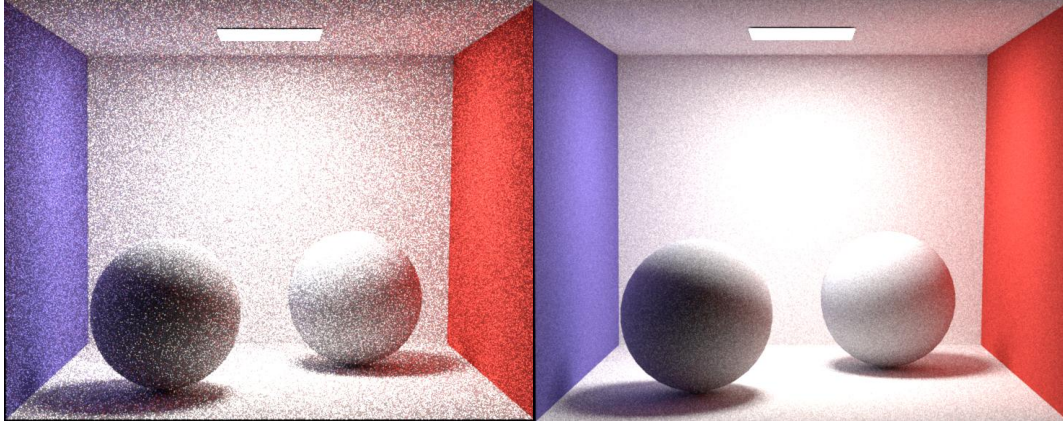


Computed ambient occlusion (left) vs. natural ambient occlusion (right)

The render on the left was produced without the indirect-lighting term and is therefore darker, making the comparison more difficult. However, it can be observed that the ambient occlusion from the previous practice is much more pronounced in all the corners of the room, regardless of whether they are directly illuminated. This produces a slightly unnatural appearance, unlike the occlusion obtained with the path tracer using NEE, which is mainly noticeable in the upper part of the scene and exhibits greater visual consistency.

Experiment 3 – Next Event Estimation and Multiple Importance Sampling

The results obtained with the path tracer using NEE were studied without applying MIS to indirect rays that intersect an emitter, in order to demonstrate the need for MIS. The results obtained were as follows:



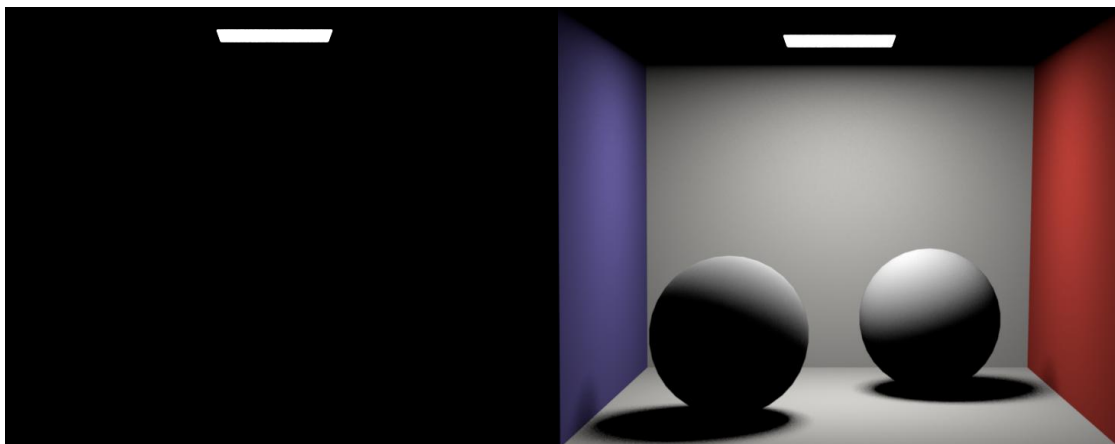
NEE without MIS, 32 spp (left) vs. 512 spp (right)

Multiple Importance Sampling is a technique used to reduce the variance of the estimator, rather than being physically necessary. Therefore, without MIS, the algorithm would eventually converge to the same solution, but this would require a much larger number of samples per pixel. In this case, it can be observed that the noise caused by the lack of MIS results from an indirect ray reaching the emitter and generating the same event as the direct NEE ray cast from the same point. As a result, the returned radiance is greater than it should be.

Experiment 4 – Bounce Limit

In this fourth experiment, the maximum number of bounces in the path tracer was limited, both with and without NEE, in order to compare the differences.

Primary ray only:

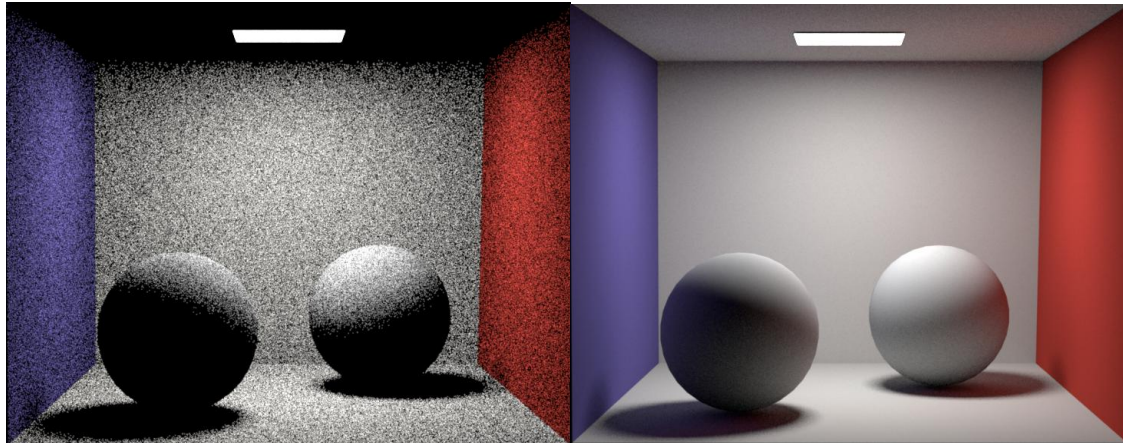


No bounces; without NEE (left) vs. with NEE (right); 128 spp

Without NEE, since indirect lighting depends on generating a new outgoing ray, the lack of bounces causes the entire scene to appear black except for the emitter. In contrast, in the

NEE implementation, the result is practically identical to that obtained in the previous practice when sampling lights for direct lighting, since, as can be seen in the left render, the indirect-lighting contribution is zero.

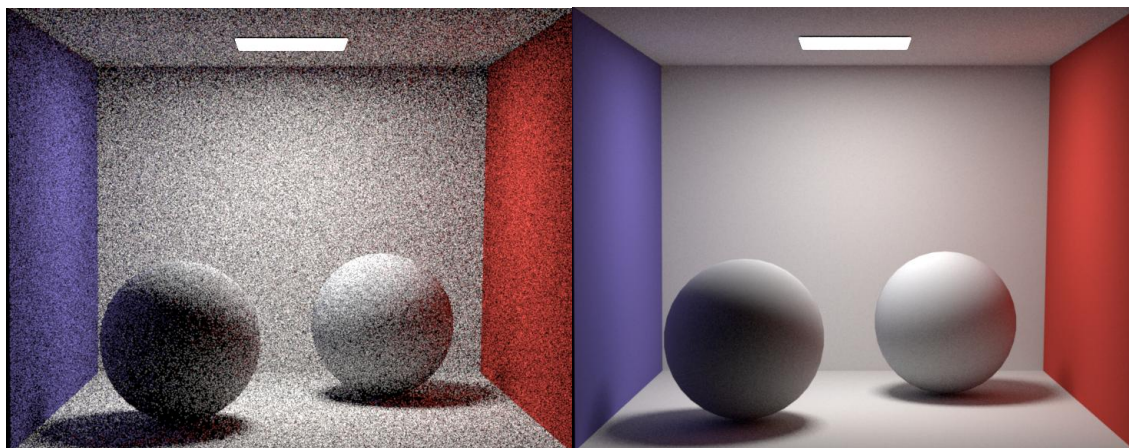
One bounce:



One-bounce limit; without NEE (left) vs. with NEE (right); 128 spp

A single bounce is sufficient to begin seeing the scene when NEE is not used. However, the resulting image is identical to that from the first practice of the course, in which only direct lighting was computed using material sampling. With NEE, on the other hand, a single bounce is already sufficient to begin capturing phenomena such as *color bleeding*, since the algorithm returns the direct lighting from the walls even though no additional bounces are generated.

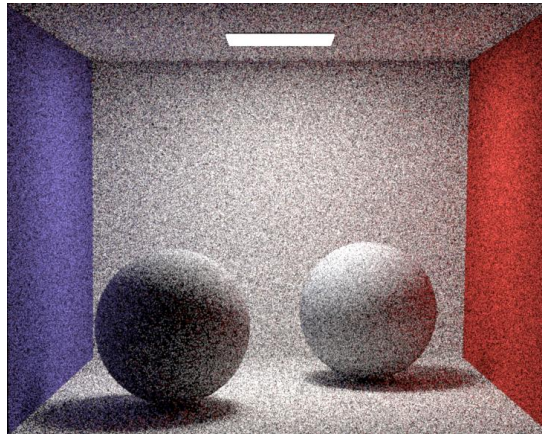
Two-bounce limit:



Two-bounce limit; without NEE (left) vs. with NEE (right); 128 spp

In this case, indirect color bleeding begins to become visible, although there is still a considerable amount of noise because it is unlikely for rays to bounce from the walls to reach the emitter. In the NEE implementation, however, there are barely any differences when increasing the bounce limit beyond this point. This makes sense because, in most cases, the ray resulting from the third or fourth bounce has very little energy and is, in fact, terminated by Russian roulette in most cases a couple of iterations later.

Finally, increasing the bounce limit for indirect lighting alone reduces the noise, which is particularly noticeable in areas affected by color bleeding:

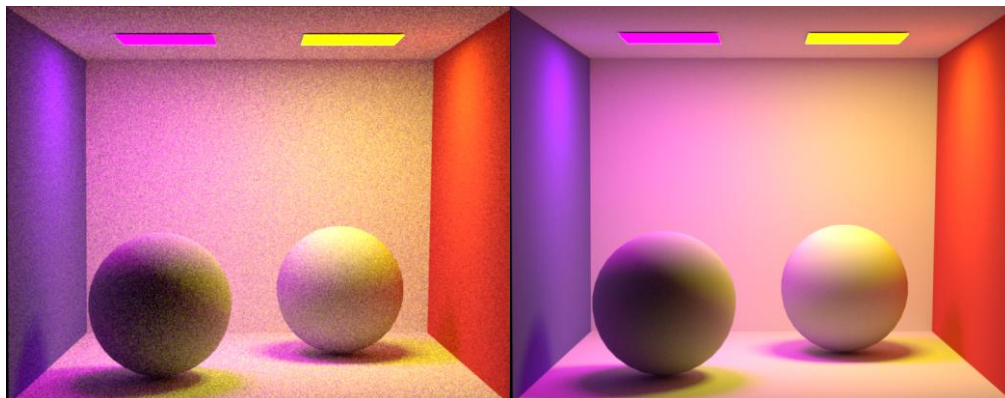


Three-bounce limit without NEE; 128 spp

Experiment 5 – Behavior in Different Scenes

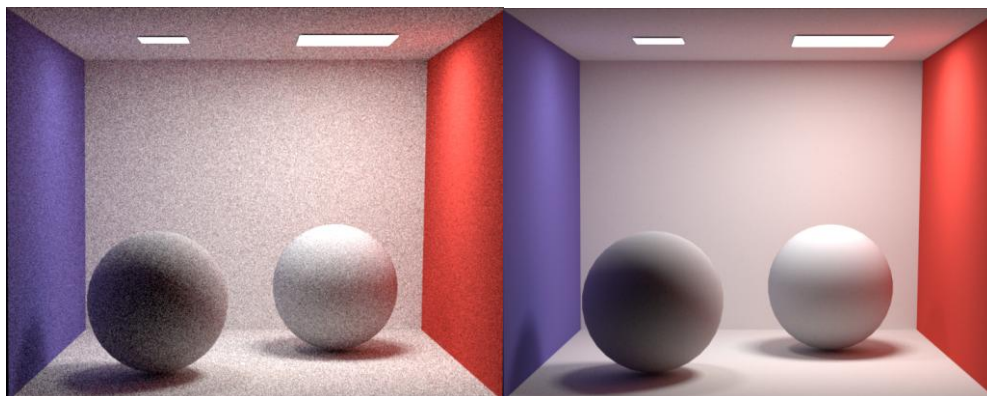
As in the previous practice, the implementations were tested on different scenes with different types of lighting, other objects, and emitters of different sizes.

For a *Cornell box* with two lights of different colors, the results were:



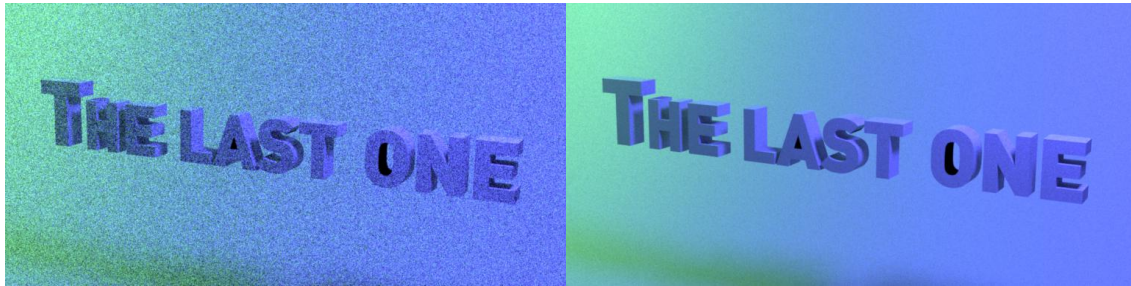
Indirect lighting only (left) vs. direct + indirect lighting (right); 512 spp

For a *Cornell box* with two lights of the same color but different sizes:



Indirect lighting only (left) vs. direct + indirect lighting (right); 512 spp

For the *motto-diffuse* scene:



Indirect lighting only (left) vs. direct + indirect lighting (right); 512 spp

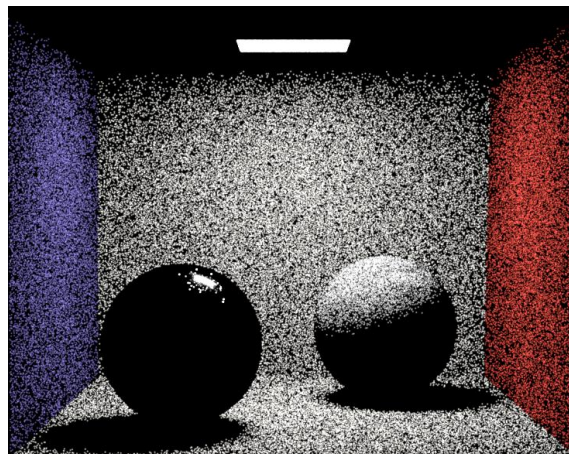
The results obtained are consistent with both the theory and the experiments and results obtained in the previous practices of the course.

Experiment 6 – Effects of MIS on a Microfacet-Based BRDF

To study the effects of MIS on materials that are neither purely diffuse nor purely specular, the previously implemented BRDF was used. The implementation from the previous practice, located in *direct.cpp*, was used for the experiment, as it already includes mechanisms that allow the different sampling strategies to be switched between trivially.

The tests were performed in a *Cornell box* containing two spheres: the left sphere has a predominantly specular component (low α), while the right sphere has a predominantly diffuse component (high α).

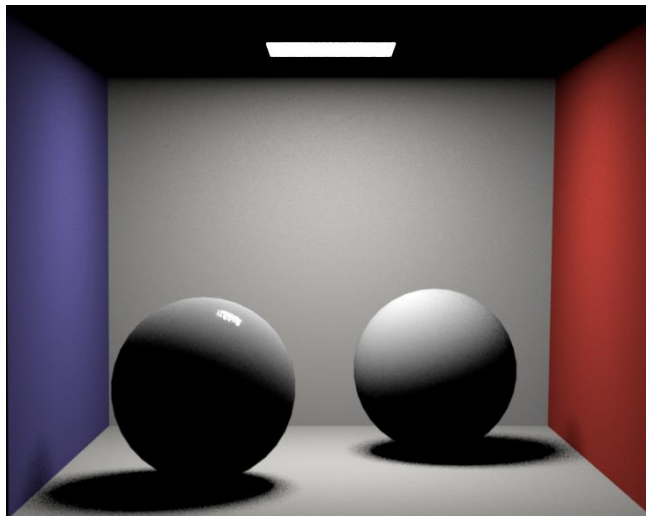
For material sampling:



Material sampling with microfacet-based BRDF materials; 32 spp

It can be observed that the area with the least noise is in the specular lobe of the left sphere. This is because material sampling is much more representative for specular reflections than for diffuse ones, as it concentrates the probability around directions close to the reflection vector.

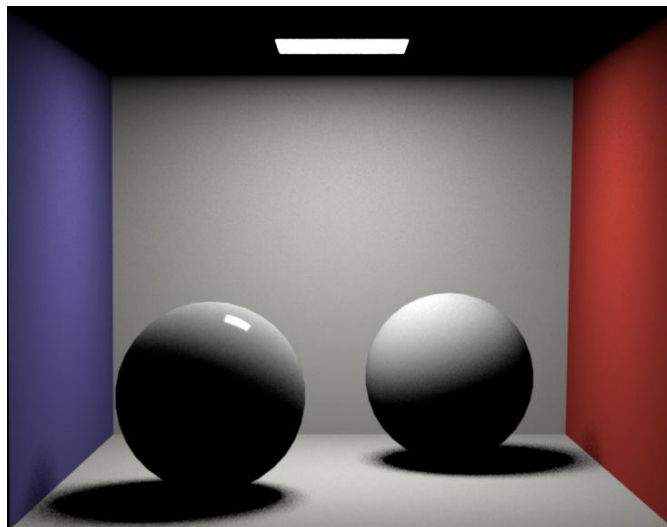
For light sampling:



Light sampling with microfacet-based BRDF materials; 32 spp

In this case, the opposite occurs: most of the noise is concentrated in the specular reflection of the left sphere, because for specular materials the probability that the sampled ray coincides with the specular lobe is low.

Finally, with MIS:



MIS with microfacet-based BRDF materials; 32 spp

This final render demonstrates the importance of using MIS to reduce variance, being particularly useful when a material cannot be sampled accurately using a single sampling strategy. MIS is especially important for scenes containing several different types of materials, reducing variance and consequently allowing the algorithm to converge to less noisy renders with fewer samples per pixel.